LangChain и tool calling

01 Что проходит по цепочке

Запрос **«Можно вернуть заказ №42?»** проходит через три шага. Смотри на тип данных после каждого.

Шаг	Вход → выход
Шаблон	`ChatPromptTemplate` подставляет вопрос и, если предусмотрено, историю. На выходе `ChatPromptValue` с сообщениями и ролями.
Модель	`ChatModel` читает сообщения и возвращает `AIMessage` с текстом или запросами инструментов.
Парсер	`StrOutputParser` извлекает текст из финального `AIMessage`.

```
# Обычный ответ текстом
chain = prompt | model | StrOutputParser()
text = chain.invoke({"question": question})
```

LCEL соединяет шаги оператором `|`. Выход слева становится входом справа. Вся цепочка тоже **Runnable**, то есть её можно запускать через тот же интерфейс, что и отдельный шаг.

Разобрать типы на примере урока.

02 Invoke, batch и stream

Вызов	Когда нужен
`invoke()`	Один вопрос → один результат. Удобно посмотреть, что вернул каждый шаг.
`batch()`	Список вопросов → список результатов. Например, проверить 10 заявок за один запуск кода.
`stream()`	Один вопрос → части одного ответа по мере готовности. Например, печатать текст в чате.

`batch` обрабатывает несколько независимых входов. Это общий интерфейс LangChain. Количество сетевых запросов зависит от реализации модели.

Чтобы видеть поток в `stream`, поддержка потоковой выдачи нужна и на последующих шагах. Шаг, который сначала собирает весь вход, задержит выдачу.

03 Схема ответа и инструмент

Задача	Механизм
Получить поля `order_id`, `eligible`, `reason`	**Structured output** задаёт схему финального ответа.
Узнать возраст заказа №42	**Tool calling** даёт модели описание `get_order`. Модель запрашивает вызов, приложение выполняет его.

В LangChain схему можно передать через `with_structured_output`. **Валидация** проверяет, что поля есть и подходят по типам. `eligible=true` может пройти схему, даже если по регламенту возврат запрещён. Факты проверяем по заказу и найденному правилу.

У аргументов инструмента своя схема. Перед вызовом код проверяет имя, параметры и доступ к заказу.

Повторить схемы и контракты.

LangChain и tool calling

04 Кто вызывает функцию

```
AIMessage → get_order(order_id=42)
            id вызова = call_1
Код → выполняет get_order(42)
ToolMessage → 40 дней, не использован
              tool_call_id = call_1
Модель → читает результат, решает дальше
```

`model.bind_tools(tools)` передаёт модели описания функций. Код читает `AIMessage.tool_calls`, находит разрешённую функцию и выполняет её. Результат добавляется как **ToolMessage** с тем же `tool_call_id`.

Заказ даёт факты о покупке. Если нужно узнать правило, модель запрашивает `search_docs`. Когда запросов инструментов больше нет, приложение может извлечь финальный текст парсером.

Посмотреть полный цикл.

05 Что модель получает снова

Так растёт список сообщений в учебном запуске. S - инструкция, U - вопрос, A - запрос инструмента от модели, T - его результат.

Запрос к модели	Что в списке
1. Выбрать первый шаг	`S, U` - 2 сообщения
2. После проверки заказа	`S, U, A1, T1` - 4 сообщения
3. После поиска правил	`S, U, A1, T1, A2, T2` - 6 сообщений

Да, код повторно передаёт накопленные сообщения. В третьем вызове модель видит и возраст заказа из `T1`, и найденное правило из `T2`. Три чанка внутри `T2` могут занимать много токенов, хотя это одно сообщение.

В длинном диалоге приложение выбирает, какую историю включать. Пара запроса и результата инструмента должна оставаться связанной.

06 Как проверить поиск агента

`search_docs` ищет чанки. **Векторный поиск** сравнивает эмбеддинги. **Гибридный** объединяет векторную выдачу и BM25, который помогает с точным кодом вроде `ERR-4031`.

RRF объединяет списки по позициям. За место `rank` чанк получает `1 / (60 + rank)` в этом примере. Вклады из списков складываются. Первое место даёт `1/61`, второе - `1/62`.

Golden set - вопросы с заранее отмеченными релевантными чанками. Для одного вопроса нужны A и B. Поиск вернул A, C, D.

Метрика	Расчёт
Precision@3	Из трёх найденных подходит один → `1/3`
Recall@3	Из двух нужных нашли один → `1/2`

Повторить RRF · Повторить метрики поиска.